

LeetCode編碼：

2368

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔



題目 (含解釋&說明)



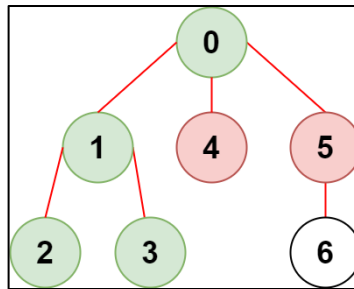
給定有 n 個節點（ $0 \sim n-1$ ）的無向樹，以及設一個 **restricted** 陣列列出被封鎖的節點。

從 0 出發，不能經過也不能停留在任何被封鎖的節點，求最多能到達幾個節點（包含起點）。

- 以 $n = 7$ ，**edge** 為 $[[0,1],[1,2],[3,1],[4,0],[0,5],[5,6]]$ ，**restricted** = $[4, 5]$ 為例。

從 0 出發，節點 4 被封鎖不能走；節點 5 被封鎖，從 0 往 5 這條路也不能走，連帶使得 6 也無法到達。

可以走的路徑是 $0 \rightarrow 1 \rightarrow 2$ 與 $0 \rightarrow 1 \rightarrow 3$ ，因此能到達的節點為 0、1、2、3，共 4 個。



- restricted** 陣列應先轉成 **set** 確保每次查詢是否被封鎖的時間複雜度為 $O(1)$ ，若用 **list** 則每次查詢為 $O(r)$ （ r 為 **restricted** 大小），複雜度會上升。

理論上不需要 **visited** 集合就能保證不重複走訪，但仍建議加入。



- 解法一：BFS

- 用廣度優先搜尋（BFS），從 Vertice 0 出發，每次從Queue取出一個節點，將其所有未被封鎖且未走訪過的 Vertices加入Queue 繼續探索。

BFS 能逐層擴散，只要在加入Queue時同時檢查是否被封鎖，被封鎖的節點就永遠不會進入Queue，後方的所有節點也自然會被隔絕。

- 建立adjacency list，因為題目給的是邊的列表，若直接用邊列表查詢某 Vertice 的所有連結 Vertice 需要 $O(e)$ 的線性掃描，而鄰接表能在 $O(\text{degree})$ 時間內直接取得，整體走訪效率為 $O(n + e)$ 。
- 優點是直觀，對應從起點出發能走到哪裡的題意。缺點是需要額外建立鄰接表與 visited 集合，空間複雜度為 $O(n + e)$ 。



- 解法二： **Union-Find**

- 與其模擬從 **Vertex 0** 出發的走訪過程，不如直接建立「哪些節點彼此相連」的連通關係，最後查詢節點 **0** 所在的連通分量大小即為答案。
- 做法是遍歷所有邊，只對兩端都不在 **restricted** 中的邊執行 **union** 操作，將兩個節點合併到同一個連通分量。

只要邊的任何一端是被封鎖的節點，這條邊就不該成為可走訪路徑的一部分，封鎖節點兩側的連通性自然被切斷。

union 操作完成後，用 **find(0)** 查詢節點 **0** 的根節點，再回傳該連通分量的大小。

- **Union-Find** 使用 **Path Compression** 與 **Union by Rank/Size** 後，每次呼叫 **find** 與 **union** 的時間複雜度接近 **$O(1)$** （ **$O(\alpha(n))$** ， α 為反阿克曼函數，可視為常數），整體時間複雜度為 **$O(n)$** 。
- 優點是一旦 **Union-Find** 結構建好，查詢任意兩節點是否連通、或任意節點的連通分量大小都是 **$O(1)$** ，在需要多次查詢的延伸情境下比 **BFS** 更有優勢。

缺點是 **Union-Find** 的實作比 **BFS** 複雜，且對單次查詢的需求大材小用。



- 被封鎖的節點會切斷樹中的連通性，任何必須經過封鎖節點才能到達的節點，無論距離起點多近，都永遠無法到達。

BFS 透過遇到封鎖節點就不加入佇列實現切斷的效果；

Union-Find 則透過不對含封鎖節點的邊做合併達到同樣效果。

兩種方法本質上都是在建立可到達的連通子圖，差別只在於建立的方式是動態走訪還是靜態合併。

方法	Time	Space
BFS	$O(n+e)$	$O(n+e)$
DFS	$O(n+e)$	$O(n+e)$
Union-Find	$O(n \times \alpha)$	$O(n)$



```

1 class Solution(object):
2     from collections import defaultdict, deque
3     def reachableNodes(self, n, edges, restricted):
4         # 轉成 set 讓每次查詢是否被封鎖的時間為 O(1)
5         # 若用 list 則每次查詢為 O(r)，整體複雜度上升
6         rs = set(restricted)
7         graph = defaultdict(list)
8         for u, v in edges:
9             graph[u].append(v)
10            graph[v].append(u)
11        # visited 避免無向邊導致來回重複走訪
12        visited = {0}
13        queue = deque([0])
14        count = 0
15        while queue:
16            node = queue.popleft()
17            count += 1
18            for nb in graph[node]:
19                # 在加入佇列時同時檢查封鎖與走訪，避免重複處理
20                if nb not in visited and nb not in rs:
21                    visited.add(nb)
22                    queue.append(nb)
23        return count
24

```

```

1 # 解法二：Union-Find
2 class UF:
3     def __init__(self, n):
4         self.p = list(range(n))
5         self.sz = [1] * n
6
7     def find(self, x):
8         if self.p[x] != x:
9             # 路徑壓縮：讓沿途所有節點直接指向根
10            # 下次查詢同一路徑上的節點時直接 O(1)
11            self.p[x] = self.find(self.p[x])
12        return self.p[x]
13
14    def union(self, x, y):
15        px, py = self.find(x), self.find(y)
16        if px == py:
17            return
18        # 小樹接到大樹下，避免樹退化成鏈結串列導致 find 變慢
19        if self.sz[px] < self.sz[py]:
20            px, py = py, px
21        self.p[py] = px
22        self.sz[px] += self.sz[py]
23
24 class Solution(object):
25     def reachableNodes(self, n, edges, restricted):
26         rs = set(restricted)
27         uf = UF(n)
28         for u, v in edges:
29             # 只對兩端都不被封鎖的邊做合併
30             # 任何一端被封鎖，這條邊就不應成為可走訪路徑的一部分
31             if u not in rs and v not in rs:
32                 uf.union(u, v)
33         # 查詢節點 0 所在連通分量的大小即為答案
34         return uf.sz[uf.find(0)]

```



- 結果證明：

data structure

Submit

Python

Auto

Code

```

1 class Solution(object):
2     from collections import defaultdict, deque
3     def reachableNodes(self, n, edges, restricted):
4         # 轉成 set 讓每次查詢是否被封鎖的時間為 O(1)
5         # 若用 list 則每次查詢為 O(n)，整體複雜度上升
6         rs = set(restricted)
7         graph = defaultdict(list)
8         for u, v in edges:

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n = 7

edges = [[0,1], [1,2], [3,1], [4,0], [0,5], [5,6]]

restricted = [4,5]

Output

4

Expected

4

data structure

Submit

Python

Auto

Code

```

1 # 解法二: Union-Find
2 class UF:
3     def __init__(self, n):
4         self.p = list(range(n))
5         self.sz = [1] * n
6
7     def find(self, x):
8         if self.p[x] != x:
9             # 路徑壓縮：讓沿途所有節點直接指向根
10            # 下次查詢同一路徑上的節點時直接 O(1)
11            self.p[x] = self.find(self.p[x])
12        return self.p[x]
13

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n = 7

edges = [[0,1], [1,2], [3,1], [4,0], [0,5], [5,6]]

restricted = [4,5]

Output

4

Expected

4



LeetCode編碼：

1584

系所： 北聯(北醫) 生物醫學工程學系
學號： B812111001
姓名： 張博翔

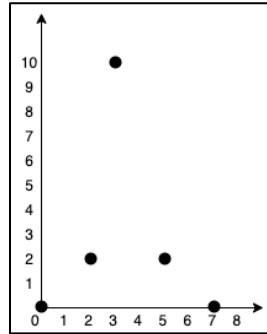




給定平面上 n 個點的座標陣列 points ，任意兩點之間的 cost 定義為它們的 Manhattan Distance： $|x1 - x2| + |y1 - y2|$ 。

求將所有點連接成一個連通網路的 Minimum cost，求這 n 個點的最小生成樹（Minimum Spanning Tree，MST）。

- 以 $\text{points} = [[0,0],[2,2],[3,10],[5,2],[7,0]]$ 為例，最優的連接方式是選擇成本最低的 4 條邊（ $n-1$ 條）將 5 個點全部連通，任何額外的邊只會增加成本而不增加連通性。最終最小總成本為 20。



- 這道題是 Dense Graph，任意兩點之間都有邊，邊的總數為 $n \times (n-1) / 2$ ，約為 $O(n^2)$ 。
與稀疏圖不同：在稀疏圖上 Prim 搭配 Heap 的 $O((n+e) \log n)$ 是好選擇，但在稠密圖上邊數本身就是 $O(n^2)$ ，使用 Heap 反而多了 $\log$ 的開銷；Prim 搭配線性掃描的 $O(n^2)$ 在稠密圖效果較佳。
MST 恰好需要 $n-1$ 條邊，選到第 $n-1$ 條邊時即可提前結束。



- 解法一： **Kruskal + Union-Find**
- **Kruskal** 的思路是從邊出發：將所有可能的邊按成本由小到大排序，依序嘗試加入每條邊，若這條邊連接的兩個端點尚未連通且加入後不會形成 **cycle** 就將它加入 **MST**；若已經連通或加入後會形成 **cycle** 就跳過。重複直到 **MST** 中有 $n-1$ 條邊為止。
- 判斷兩個節點是否已連通，使用 **Union-Find**。
每次加入一條邊就執行 **union** 操作，合併兩個節點所在的連通分量；每次判斷是否會形成 **cycle** 就執行 **find** 操作，看兩個節點的根節點是否相同。
使用路徑壓縮與按大小合併後，**Union-Find** 的每次操作接近 **$O(1)$** 。
- 瓶頸在於列舉所有邊並排序的步驟。 n 個點共有 $n \times (n-1) / 2$ 條邊，排序需要 $O(n^2 \log n)$ ，空間上也需要 $O(n^2)$ 存放所有邊，在稠密圖上這是明顯的缺點。



- 解法二：Prim 無 Heap
- Prim 的思路則是從 Vertice 出發：從任意一個節點開始，每次將「距離當前 MST 最近的節點」加入 MST，並更新其他節點到 MST 的最短距離，重複 n 次直到所有節點都加入。
- 設一個 `min_dist` 陣列，`min_dist[i]` 表示節點 i 加入 MST 的最小成本。
 初始節點 0 的 `min_dist` 為 0，其餘節點設為無限大。
 每輪從尚未加入 MST 的節點中線性掃描找出 `min_dist` 最小的節點 u ，將其加入 MST，再遍歷所有未加入的節點，用 u 到它們的距離更新 `min_dist`。
- 不使用 Heap 而改用線性掃描的原因是每輪都需要更新幾乎所有未加入節點的 `min_dist`（因為是稠密圖，每個新加入的節點都與所有其他節點相連），更新的總次數本來就是 $O(n^2)$ 。
 若使用 Heap，每次更新都需要在 Heap 中進行調整，反而多了 $\log n$ 的常數倍；線性掃描找最小值是 $O(n)$ ，與更新步驟合併後整體仍是 $O(n^2)$ 。



方法	Time	Space
Kruskal	$O(n^2 \log n)$	$O(n^2)$
Prim+Heap	$O(n^2 \log n)$	$O(n)$
Prim 無Heap	$O(n^2)$	$O(n)$



```

1 # 解法一: Kruskal + Union-Find
2 class UF:
3     def __init__(self, n):
4         self.p = list(range(n))
5         self.sz = [1] * n
6
7     def find(self, x):
8         if self.p[x] != x:
9             self.p[x] = self.find(self.p[x])
10        return self.p[x]
11
12    def union(self, x, y):
13        px, py = self.find(x), self.find(y)
14        if px == py:
15            # 兩端已連通, 加入這條邊會形成環, 跳過
16            return False
17        if self.sz[px] < self.sz[py]:
18            px, py = py, px
19        self.p[py] = px
20        self.sz[px] += self.sz[py]
21        return True
22
23 class Solution(object):
24     def minCostConnectPoints(self, points):
25         """
26         :type points: List[List[int]]
27         :rtype: int
28         """
29        n = len(points)
30        edges = []
31        for i in range(n):
32            for j in range(i+1, n):
33                d = abs(points[i][0]-points[j][0]) + abs(points[i][1]-points[j][1])
34                edges.append((d, i, j))
35        # 按成本排序, 確保每次加入的都是當前最便宜的邊
36        edges.sort()
37        uf = UF(n); total = 0; cnt = 0
38        for cost, u, v in edges:
39            if uf.union(u, v):
40                total += cost; cnt += 1
41            # MST 恰好需要 n-1 條邊, 達到後提前結束
42            if cnt == n-1:
43                break
44        return total

```

```

1 class Solution(object):
2     # 解法二: Prim 無 Heap (稠密圖最優  $O(n^2)$ )
3     def minCostConnectPoints(self, points):
4         n = len(points)
5         in_mst = [False] * n
6         # min_dist[i] 表示節點 i 加入 MST 的當前最小成本
7         # 初始為無限大, 表示尚未找到任何連接方式
8         min_dist = [float('inf')] * n
9         # 從節點 0 開始, 成本為 0 因為它是起點不需要連接任何人
10        min_dist[0] = 0
11        total = 0
12        for _ in range(n):
13            # 線性掃描找未加入 MST 中成本最小的節點
14            # 不用 Heap 是因為稠密圖每輪都要更新幾乎所有節點
15            # Heap 的維護成本反而比線性掃描多出  $\log n$  的倍數
16            u = min(
17                (v for v in range(n) if not in_mst[v]),
18                key=lambda v: min_dist[v]
19            )
20            in_mst[u] = True
21            total += min_dist[u]
22            for v in range(n):
23                if not in_mst[v]:
24                    d = abs(points[u][0]-points[v][0]) + abs(points[u][1]-points[v][1])
25                    # 若 u 到 v 的距離比目前已知的最小成本更小, 更新
26                    # 這樣確保每個節點只以最便宜的方式加入 MST
27                    if d < min_dist[v]:
28                        min_dist[v] = d
29        return total
30

```



- 結果證明：

data structure

Submit

Premium

Code

Python Auto

```

1 # 解法一: Kruskal + Union-Find
2 class UF:
3     def __init__(self, n):
4         self.p = list(range(n))
5         self.sz = [1] * n
6
7     def find(self, x):
8         if self.p[x] != x:
9             self.p[x] = self.find(self.p[x])
10        return self.p[x]
11
12    def union(self, x, y):
13        px, py = self.find(x), self.find(y)
14        if px == py:
15            # 兩端已連通，加入這條邊會形成環，跳過
16            return False
17        if self.sz[px] < self.sz[py]:
18            px, py = py, px
19        self.p[py] = px
20        self.sz[px] += self.sz[py]
21        return True
22

```

Testcase 1 Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

points =

[[0,0], [2,2], [3,10], [5,2], [7,0]]

Output

20

Expected

20

data structure

Submit

Premium

Code

Python Auto

```

1 class Solution(object):
2     # 解法二: Prim 無 Heap (稠密圖最優 O(n^2))
3     def minCostConnectPoints(self, points):
4         n = len(points)
5         in_mst = [False] * n
6         # min_dist[i] 表示節點 i 加入 MST 的當前最小成本
7         # 初始為無限大，表示尚未找到任何連接方式
8         min_dist = [float('inf')] * n
9         # 從節點 0 開始，成本為 0 因為它是起點不需要連接任何人
10        min_dist[0] = 0
11        total = 0
12        for _ in range(n):
13            # 線性掃描找未加入 MST 中成本最小的節點
14            # 不用 Heap 是因為稠密圖每輪都要更新幾乎所有節點
15            # Heap 的維護成本反而比線性掃描多出 log n 的倍數
16            u = min(
17                (v for v in range(n) if not in_mst[v]),
18                key=lambda v: min_dist[v]
19            )
20            in_mst[u] = True
21            total += min_dist[u]
22            for v in range(n):

```

Testcase 1 Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

points =

[[0,0], [2,2], [3,10], [5,2], [7,0]]

Output

20

Expected

20